

# Analytics for a Better World

Predicting while optimizing: mobile clinic routing with learned constraints

Joaquim Gromicho

joint work with Mayukh Ghosh and Chintan Amrit

Amsterdam Business School, University of Amsterdam

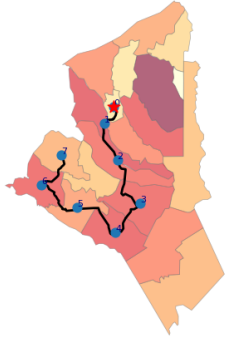
Chapter 6 of Mayukh Ghosh's PhD thesis, *Domain Driven Analytics*



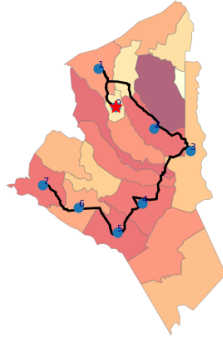
UNIVERSITY OF AMSTERDAM

# Same history, same solver, same number of stops

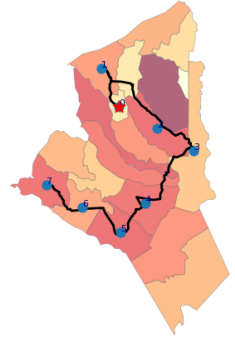
cannibalisation: static (I2)  
8 stops, 56 km, promised 1892, realised 1374



cannibalisation: learned MLP (I3)  
8 stops, 79 km, promised 1664, realised 1621



cannibalisation: oracle  
8 stops, 79 km, promised 1653, realised 1630



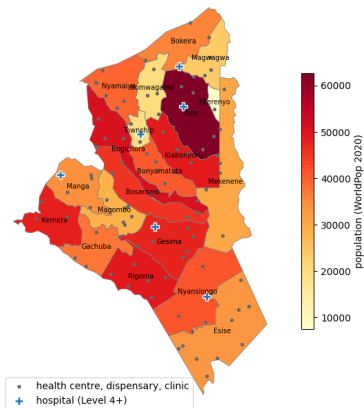
Left: 1892 doses promised, 1374 delivered. Middle: 1664 promised, 1621 delivered. Why?

## The setting: one mobile clinic, many wards

- AMREF Health Africa ran mobile clinics for COVID-19 vaccination in Kenya (Ghosh, Amrit, Gromicho; thesis Chapter 6).
- One clinic, a planning period of a few weeks, at most a handful of stops.
- Vaccines are collected at a referral hospital, so the route is an open path from town.
- Which wards to visit, and in what order?

Today: Nyamira County, 20 wards, 745 thousand people, 897 km<sup>2</sup>.

Nyamira County: wards, population and health facilities



# Predict-then-optimize, and its hidden assumption

- 1 Predict demand  $\hat{d}_i$  per ward from population, unvaccinated share, accessibility.
- 2 Solve a prize-collecting routing problem that rewards  $\hat{d}_i$  at visited wards.

## Hidden assumption

Demand at ward  $i$  does not depend on which other wards the clinic visits.

Two reasons it does:

- **Cannibalisation:** people near a visited ward get their dose there; a later visit next door finds less demand.
- **Mobilisation:** health volunteers, market days and word of mouth raise turnout around a visited ward.

# Spillover in one formula

Let  $y_j = 1$  if the clinic visits ward  $j$ , and  $d_{ij}$  the road distance.

$$E_i(y) = \min\left(1, \sum_{j \neq i} e^{-d_{ij}/R} y_j\right) \qquad D_i(y) = b_i(1 + \gamma E_i(y)) \varepsilon_i$$

- $E_i$ : exposure of ward  $i$  to visits nearby, range  $R$  (10 km today); one adjacent visit gives about 0.6, two saturate it.
- $b_i$ : base demand if visited alone; grows with population, unvaccinated share, distance to the nearest facility.
- $\gamma < 0$  cannibalisation,  $\gamma > 0$  mobilisation,  $\gamma = 0$  the textbook case.
- The clinic administers at most 250 doses per visit.

Real turnout data would replace the simulator; the structure of the question stays.

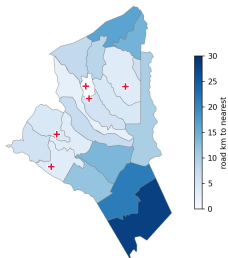
# Open data, no API keys

| Ingredient                   | Source                                                                  |
|------------------------------|-------------------------------------------------------------------------|
| Ward boundaries              | <a href="#">GADM</a> 4.1, level 3                                       |
| Population per ward          | <a href="#">WorldPop</a> 2020, 1 km, UN-adjusted                        |
| Road network                 | <a href="#">OpenStreetMap</a> through <code>osmnx</code>                |
| Health facilities, geocoded  | <a href="#">Maina et al. (2019)</a> , <i>Scientific Data</i> 6:134, CC0 |
| Road distances between wards | <code>pandana</code> contraction hierarchies, 20 by 20 in milliseconds  |
| Demand                       | simulator above, both signs of $\gamma$                                 |
| Optimization                 | Gurobi with <a href="#">gurobi-machinelearning</a> , free licence size  |

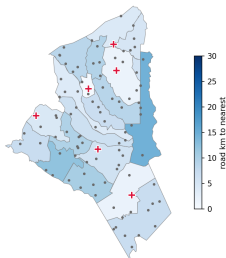
Nyamira has 98 geocoded public facilities in this list. Only its 6 hospitals (Level 4 and above) store vaccines, so accessibility is the road distance to the nearest of those.

# Same count, different facilities: why the source matters

OpenStreetMap: 6 tagged facilities



Maina et al. 2019: 6 hospitals of 98 facilities



The conclusion survives the change of source; the route and the wards it favours do not. Maina et al. is public sector only, from about 2018.

|                              | OSM        | Maina et al. |
|------------------------------|------------|--------------|
| Facilities in Nyamira        | 6          | 98           |
| Used as vaccine points       | 6          | 6            |
| Of those, real hospitals     | 2          | 6            |
| Distinct sites               | 5          | 6            |
| Mean road km to nearest      | 9.0        | 6.8          |
| Worst ward, road km          | 28.2       | 14.6         |
| Static: promised, delivered  | 1889, 1331 | 1892, 1374   |
| Learned: promised, delivered | 1551, 1530 | 1664, 1621   |

- OSM tags two dispensaries and a health centre as hospitals and lists one site twice.
- Only one site is common to both: the county hospital in Nyamira town.
- Ward accessibility rankings are unrelated, rank correlation 0.06.

# Learning a demand model that contains the decision

Where does a training set with  $y$  in it come from? Replay the history of past deployments.

| period, ward | log pop | unvaccinated | inaccessibility | exposure $E_i$ | turnout $D_i$ |
|--------------|---------|--------------|-----------------|----------------|---------------|
| 1, Esise     | 10.4    | 0.41         | 1.00            | 0.31           | 361           |
| 1, Gesima    | 10.8    | 0.52         | 0.49            | 1.12           | 275           |
| 2, Gesima    | 10.8    | 0.52         | 0.49            | 0.08           | 560           |
| $\vdots$     |         |              |                 |                |               |

- 1500 random deployments of 3 to 10 wards, 30 000 rows.
- Two models: linear regression (transparent, mis-specified) and an 8-unit ReLU network, fitted with L-BFGS in seconds.
- In practice: past periods' routes plus ticketed turnout, the feedback loop AMREF asked for.

# The routing model

Wards  $N$ , depot  $s$ , arcs  $A$ , road distance  $c_{ij}$ , cost  $\mu$  per km, stop limit  $K$ , capacity  $C$  per visit, spillover weights  $w_{ij}$ , ward features  $f_i$ ,  $M = 4C$ .  $x_{ij}, y_i \in \{0, 1\}$ : drive  $i \rightarrow j$ , visit  $i$ ;  $E_i \in [0, 1]$  exposure;  $\hat{d}_i \geq 0$  modelled demand;  $z_i \in [0, C]$  doses delivered.

$$\begin{aligned}
 \max \quad & \sum_{i \in N} z_i - \mu \sum_{(i,j) \in A} c_{ij} x_{ij} \\
 \text{s.t.} \quad & y_s = 1, \sum_j x_{sj} = 1, \sum_j x_{js} = 0 && \text{path starts at the depot} \\
 & \sum_{j \neq i} x_{ji} = y_i && \forall i \neq s \\
 & \sum_{j \neq i} x_{ij} \leq y_i && \forall i \in N \\
 & \sum_{(i,j) \in A} x_{ij} = \sum_i y_i - 1, \quad \sum_i y_i \leq K \\
 & E_i = \min(1, \sum_{j \neq i} w_{ij} y_j) && \forall i \in N \\
 & \hat{d}_i \leq g(f_i, E_i) + M(1 - y_i) && \forall i \in N \quad \text{the demand model} \\
 & z_i \leq \hat{d}_i, \quad z_i \leq C y_i && \forall i \in N \\
 & \sum_{i,j \in S} x_{ij} \leq \sum_{i \in S} y_i - y_k && \forall S \subseteq N \setminus \{s\}, k \in S \quad \text{DFJ, lazy}
 \end{aligned}$$

The DFJ family is generated in a MIPSOL callback with cbLazy: follow the successors from the depot, every visited ward not reached lies on a cycle  $S$ . Details and the MTZ comparison: [MO-book, Traveling Salesman Problem](#).

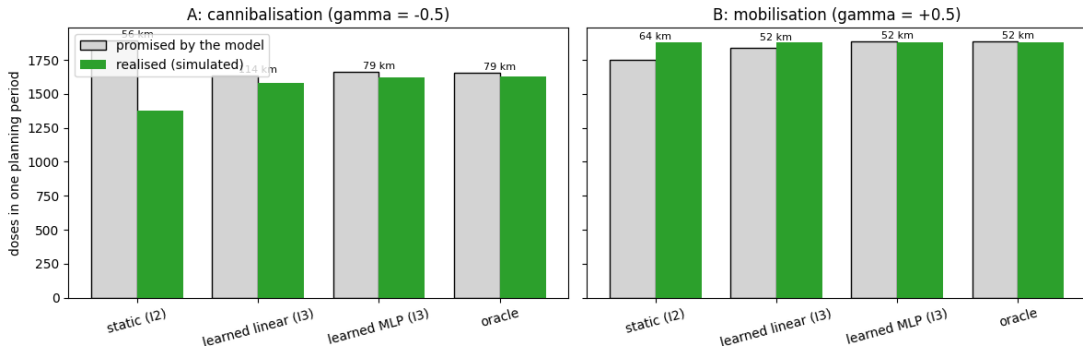
# Three ways to fill in $g$ inside the same routing model

Same path constraints, same objective; only the demand model  $g$  changes.

| Variant | Thesis analogue | The demand model $g$                                     |
|---------|-----------------|----------------------------------------------------------|
| static  | Intervention I2 | $g(f_i, 0)$ , a number fixed before solving              |
| learned | Intervention I3 | $g(f_i, E_i)$ , the fitted model embedded as constraints |
| oracle  | upper bound     | $b_i (1 + \gamma E_i)$ , the simulator's own mean        |

- Embedding: one call, `add_predictor_constr(model, g, features, output)` from `gurobi-machinelearning`.
- Size with the network: 980 variables, 404 constraints, no subtour constraint among them. Solves in about a second.
- The oracle is linear in  $y$  once  $E_i$  is in the model, so it is an exact MIP, not a simulation.

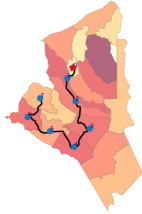
# Promised versus delivered



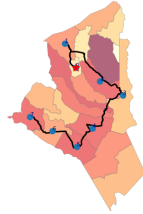
- Cannibalisation: the static plan delivers 73% of what it promises; the learned network delivers 97% and 18% more doses.
- Mobilisation: the same doses, but the static plan drives 64 km where 52 km suffice; doses per km go from 29 to 36.
- The realised-to-promised ratio is a diagnostic any programme can compute after the fact.

# The two signs pull the route in opposite directions

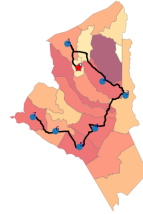
cannibalisation: static (i2)  
8 stops, 56 km, promised 1892, realised 1374



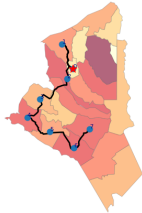
cannibalisation: learned MLP (i3)  
8 stops, 79 km, promised 1664, realised 1621



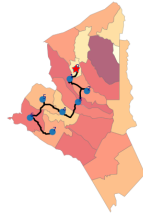
cannibalisation: oracle  
8 stops, 79 km, promised 1653, realised 1630



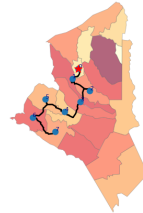
mobilisation: static (i2)  
8 stops, 64 km, promised 1750, realised 1882



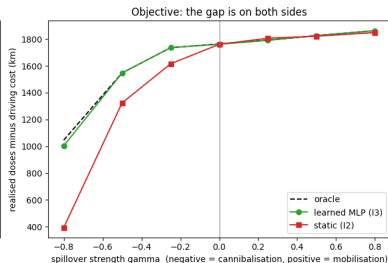
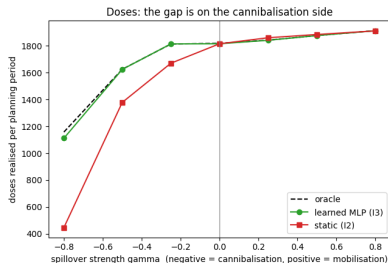
mobilisation: learned MLP (i3)  
8 stops, 52 km, promised 1887, realised 1877



mobilisation: oracle  
8 stops, 52 km, promised 1884, realised 1879



# How much does it matter? Sweep the interaction strength



- At  $\gamma = 0$  the three coincide, as they should.
- Cannibalisation costs doses; mobilisation costs kilometres. Only the objective panel shows both.
- Models are retrained per  $\gamma$ ; the learned line tracks the oracle throughout.

# Live demo: you predict, I run

The notebook is live in Colab. For each experiment: first your prediction by show of hands, then the rerun.

- ① **Range.** R\_KM from 10 to 5, then 20. Does the gap between static and learned grow or shrink?
- ② **Stops.** K\_STOPS from 8 to 4, then 12. At which end can the static planner ignore spillover?
- ③ **Model size.** HIDDEN from 8 units to 2. What happens to the model size, and to the doses delivered?

Each rerun takes a few seconds: data is cached, every solve is about a second.

Try it yourself, now or later

[github.com/gromicho/mobile\\_clinics](https://github.com/gromicho/mobile_clinics)

[Open the notebook in Colab.](#) No installation needed.

# Debrief and the open question

- The planner's error is structural, not statistical: both planners saw the same data.
- Predicting while optimizing means the prediction is a constraint, not an input.
- Keep the model small enough to embed: a linear or tree model, or a one-layer network.

## Open question for research and for practice

In a real deployment, where does the training data with decisions in it come from, and how many periods are needed before the learned constraints beat a careful static forecast?

Notebook, data and slides: [github.com/gromicho/mobile\\_clinics](https://github.com/gromicho/mobile_clinics). Thesis: M. Ghosh, *Domain Driven Analytics*, University of Amsterdam, Chapter 6.